

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	RISHIK JK	Reg. No.:	192511052
Date:	15/08/2026	Duration:	60 minutes

Code of Conduct

*I **RISHIK JK (192511052)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **RISHIK JK**

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation	Poor or insufficient documentation.

Maintenance(15 %)	complete, clear, and well organized.		with missing details.	
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

CO3-AT2-CODE REVIEW AND REPOSITORY EVALUATION

NAME: RISHIK JK

SLOT: C

REGNO :192511052

COURSE CODE: CSA1007

DATE:15/08/2026

1. Problem Analysis

Problem Statement

The Smart Renewable Battery Recycling Platform is developed to provide a centralized system for managing used battery collection, storage, transportation, and recycling activities. Traditional battery recycling processes may involve separate records and manual monitoring, which can make it difficult to track battery conditions, recycling activities, transportation details, and sustainability information efficiently.

The proposed platform organizes these activities through a single software system. It allows recycling operators to maintain battery records, monitor battery conditions, manage recycling operations, track transportation activities, receive important notifications, and generate sustainability-related reports.

Implemented Solution

The proposed solution is a web-based platform that connects the major activities involved in battery recycling. The system organizes battery information and provides different functionalities for inventory management, recycling operations, logistics tracking, hazard monitoring, and reporting.

The application follows a modular structure so that each major functionality can be developed and maintained independently. The system can also be deployed using Docker to provide a consistent application environment.

Project Objectives

- To manage battery collection and recycling activities efficiently.
- To maintain accurate battery inventory information.
- To monitor battery conditions and identify possible hazards.
- To track battery transportation and logistics activities.
- To provide notifications for important battery conditions.
- To generate useful recycling and sustainability reports.
- To improve the overall efficiency and safety of battery recycling operations.

Key Functionalities

The major functionalities of the system include:

- User authentication and access management.
- Battery inventory management.
- Battery condition monitoring.
- Hazard detection and notification.
- Recycling activity management.
- Battery collection and transportation tracking.
- Sustainability and recycling report generation.
- Centralized monitoring of recycling activities.

2. Repository Organization

The repository is organized into separate application, database, testing, and deployment components. This structure makes the project easier to understand and allows developers to locate the required files quickly.

The main application folder contains the different components responsible for handling application requests, business logic, data models, user interface pages, and static resources. The database folder contains database-related files, while the testing folder contains test cases used to validate the application. The Docker file and Docker Compose configuration are maintained at the project level because they are responsible for application deployment and container management. The requirements file contains the required software dependencies, while the README file provides information about the project.

Repository Organization Evaluation

The repository organization is considered suitable because:

- Related files are grouped together.
- Application logic is separated from database and testing components.
- The folder names clearly describe their purpose.
- Important configuration files are maintained at the project root.
- The structure supports easier debugging and maintenance.
- Developers can work on different application components independently.
- New features can be added without disturbing the complete project structure.

File Naming Evaluation

The project uses meaningful and descriptive names for folders and files. Names such as routes, services, models, database, and tests clearly communicate their purpose.

Consistent naming improves readability and helps new developers understand the repository without requiring extensive explanation.

3. Code Quality Review

Code quality is evaluated based on readability, modularity, coding standards, documentation, and maintainability.

The project follows a modular approach in which different responsibilities are separated into appropriate components. Application routes handle requests, services contain business logic, models represent data, templates manage the user interface, and tests are maintained separately.

Code Quality Strengths

- The code is organized into separate modules.
- Functions and components have specific responsibilities.
- The project structure improves readability.
- Related functionality is grouped together.
- Separating business logic improves maintainability.
- Testing files are maintained separately from application files.
- Dependencies are maintained separately in the requirements file.
- Docker configuration is separated from application logic.

Readability

The source code should use meaningful variable, function, and class names so that the purpose of each component can be understood easily. Proper indentation and consistent formatting also improve code readability.

Avoiding unnecessarily complex functions makes the application easier to understand and modify.

Modularity

The project follows modular development by separating different system responsibilities. Inventory management, recycling management, logistics, alerts, and reporting can be maintained as independent components.

This makes it easier to modify one functionality without creating unnecessary changes in other parts of the application.

Coding Standards

The project should follow consistent coding practices such as:

- Meaningful names for variables and functions.
- Consistent indentation and formatting.
- Small and focused functions.
- Proper error handling.
- Avoidance of repeated code.
- Clear comments where necessary.
- Separation of application logic from configuration.

Areas for Improvement

Although the project structure supports good code quality, further improvements can be made by increasing test coverage, improving error handling, adding more detailed documentation, and reducing repeated logic.

Complex functions should be divided into smaller functions when necessary. Regular code reviews can also help identify unused code, inefficient logic, and potential security issues.

4. Version Control Evaluation

Git is used to maintain the source code and development history of the Smart Renewable Battery Recycling Platform. Version control allows developers to track changes and collaborate without losing previous versions of the project.

The repository follows a feature-based development approach where different functionalities can be developed independently. This is useful because inventory management, recycling management, logistics, and notification features can be developed and tested separately.

Commit History Evaluation

The commit history should contain meaningful descriptions that clearly indicate the changes made during development. Examples include project setup, inventory development, recycling functionality, database integration, Docker configuration, testing, and bug fixing.

Meaningful commits make it easier to understand the development process and identify when a particular change was introduced.

Branching Strategy Evaluation

The project uses separate development and feature branches to support organized development. Feature branches allow developers to work independently without directly affecting the stable version.

This approach reduces development conflicts and allows completed features to be reviewed and tested before integration.

Version Control Strengths

- Maintains complete development history.
- Supports parallel development.
- Reduces risk to the stable version.
- Makes previous versions available when required.
- Supports team collaboration.
- Makes changes easier to track.
- Helps identify and correct development problems.

Areas for Improvement

Commit messages should always remain meaningful and consistent. Unnecessary commits should be avoided, and completed features should be reviewed before integration.

The repository should also regularly remove unused files and maintain a clean branch structure.

5. Code Testing and Validation

Testing is performed to verify that the application functions according to its requirements. The testing process focuses on the major functionalities of the Smart Renewable Battery Recycling Platform.

The application should be tested under different conditions to ensure that user actions produce the expected results.

Major Test Areas

User Authentication

The login functionality is tested using valid and invalid user information. The system should allow authorized users to access the application and reject invalid login attempts.

Battery Inventory

The inventory module is tested by adding, updating, viewing, and managing battery records. The system should correctly store and display battery information.

Battery Condition Monitoring

The system is tested using different battery condition values. It should identify important or hazardous conditions correctly and provide appropriate notifications.

Recycling Management

Recycling activities are tested by creating and updating recycling records. The system should correctly maintain the status of recycling operations.

Logistics Tracking

Collection and transportation information is tested to verify that battery movement details are recorded and displayed correctly.

Notifications

The notification functionality is tested using different battery conditions and operational events. Important conditions should generate appropriate notifications.

Report Generation

The reporting functionality is tested to verify whether recycling and sustainability information is generated correctly.

Testing Outcome

The expected testing outcome is that all major functions operate correctly and produce the expected results. Any identified errors should be corrected and retested before the final version is released.

Testing improves system reliability and helps ensure that the application meets its functional requirements.

6. Repository Documentation

Documentation plays an important role in helping developers understand, install, use, and maintain the project.

The project documentation should provide sufficient information about the system, its requirements, installation process, usage, and deployment.

README Documentation

The README file should contain:

- Project title and description.
- Problem statement.
- Project objectives.
- Major system functionalities.
- Technologies used.
- Project structure.
- Installation requirements.
- Application setup instructions.
- Usage information.
- Docker deployment information.
- Testing information.
- Future enhancements.

Installation Documentation

The installation section should clearly explain the software and dependencies required to run the application. It should also explain the required database configuration and application setup.

Usage Documentation

The usage guide should explain how users can access the system and use its major functions such as inventory management, recycling management, logistics tracking, notifications, and report generation.

Documentation Strengths

- Project purpose is clearly defined.
- Major functionalities can be documented separately.
- Repository structure provides a clear understanding of the application.
- Dependency information can be maintained in a separate file.
- Docker configuration simplifies deployment instructions.

Documentation Improvements

The documentation can be improved by providing clearer setup instructions, detailed usage examples, troubleshooting information, and updated screenshots.

Documentation should also be updated whenever new features or major changes are added to the project.

7. Code Review Findings and Recommendations

The overall code review shows that the Smart Renewable Battery Recycling Platform follows a structured approach for application development and repository management. The separation of application components, testing, database files, and deployment configuration improves maintainability.

The modular structure makes it easier to develop individual features and allows future functionality to be added without redesigning the complete application.

Major Findings

- The repository has a logical structure.
- Application responsibilities are separated into modules.
- The project supports feature-based development.
- Git provides effective version management.
- Meaningful commit practices improve project tracking.
- Testing is required for validating major application functions.
- Documentation provides important information for project maintenance.
- Docker improves deployment consistency and portability.

Recommendations for Code Quality

- Maintain consistent coding standards.
- Use meaningful names for functions and variables.
- Avoid unnecessary duplicate code.
- Improve error handling.
- Increase automated test coverage.
- Perform regular code reviews.
- Keep modules small and focused.

Recommendations for Repository Management

- Maintain a clean branch structure.
- Use meaningful commit messages.
- Remove unused files and branches.
- Keep the repository documentation updated.
- Review changes before merging them.

Recommendations for Testing

- Increase the number of automated test cases.
- Test both normal and invalid inputs.
- Perform integration testing between application and database.
- Test important functions after every major update.
- Maintain test cases for future features.

Future Enhancements

- Implement automated CI/CD.
- Add cloud deployment.
- Improve real-time battery monitoring.
- Introduce advanced hazard prediction.
- Add automated database backup.
- Improve security and access control.
- Add advanced analytics for recycling performance.

CONCLUSION

The code review and repository evaluation of the Smart Renewable Battery Recycling Platform shows that a structured development approach can significantly improve software quality and maintainability. The modular repository organization makes the application easier to understand and extend, while Git supports collaboration and effective version management. The code quality can be further improved through consistent coding standards, better error handling, increased test coverage, and regular code reviews.